

Oracle 8i Legacy Database Handled-Work Summary

Public-sharing note: this document is based on a real handled production case, but all IP addresses, server labels, schema/user names, database names, tablespace names, table names, column names, index names, file paths, job numbers, hostnames, and date references have been changed so the company's real database structure and environment details are not exposed. The technical shape of the work is preserved for learning and discussion.

Case note: this document is based on a real handled production case. The individual technical steps were not especially complex, but the risk level was high because both the operating system and database platform were very old. On this kind of legacy stack, a small wrong memory, boot, listener, or storage decision can be hard to recover from and may leave very limited rollback options.

Outcome note: after the handled work and follow-up monitoring, the database is currently running well and stable. This was a successful legacy database recovery and stabilization case.

Document date: 17/04/2026

Server: 192.0.2.50

Database/SID: LEGACYDB

Main schema/user: APP_SCHEMA

Purpose: this document is a standalone printable summary of the Oracle 8i primary production server work already handled or prepared. It includes the action date, reason, concrete settings, and the main SQL/init/OS commands used or prepared.

Executive Summary

Oracle 8i primary production server was handled across the following areas:

1. Checked and prepared fixes for ORA-03232 extent allocation risk.
2. Standardized non-SYSTEM tablespace storage and added capacity.
3. Created the TXN_DETAIL_A performance index and refreshed statistics.
4. Recorded the earlier near-2GB / 800 processes test profile from server test clone server, a clone of primary production server with the same OS and configuration, which failed to start before /3GB /USERVA=2560.
5. Applied the larger 1.5GB / 600 processes memory profile on 03/04/2026, then rolled back after the production issue on 07/04/2026.
6. Created and validated a safer 981MB / 450 processes profile.
7. Prepared and applied the 1100MB / 500 processes direction, then monitored listener/process risk.
8. Prepared Windows Server 2003 /3GB /USERVA=2560 mitigation for ORA-12500 and ORA-04030.
9. Captured post-change risk: process saturation and heavy physical reads.
10. Reviewed the high-read REPORT_DETAIL_A report query and applied the first index fix; optional follow-up indexes were not run.
11. Prepared the 1100MB / 550 processes direction for continued monitoring.

12. Captured the 17/04/2026 monitoring result showing low shared pool free memory and SQL AREA pressure.

Current operational position:

- Keep primary production server under monitoring after the 1100MB / processes = 550 direction.
- Do not increase SGA/processes further without same-time Oracle and Windows memory evidence.
- If ORA-12500 or ORA-04030 returns below the configured process limit, treat it as memory/address-space pressure and roll back toward the smaller validated profiles.

1. ORA-03232 Extent Allocation Risk Check

Action date: 01/04/2026

Reason:

- ORA-03232: unable to allocate an extent from tablespace indicates extent allocation risk.
- In this case, the check showed an extent/storage setting around 40KB, while the failing allocation required 8 Oracle blocks.
- The database block size was 8192 bytes, so the required extent was $8 * 8192 = 65536$ bytes, or 64KB.
- The immediate fix for this incident was therefore to set the relevant extent size to 64KB.

Main check SQL:

```
SELECT name, value
FROM v$parameter
WHERE name = 'db_block_size';

-- Observed block size:
-- db_block_size = 8192 bytes

-- ORA-03232 required extent:
-- 8 blocks * 8192 bytes = 65536 bytes = 64KB

-- Problem found:
-- Existing extent/default storage was around 40KB, smaller than the 64KB
allocation request.

SELECT tablespace_name,
       initial_extent,
       next_extent,
       min_extents,
       max_extents,
       pct_increase,
       extent_management,
       allocation_type
FROM dba_tablespaces
ORDER BY tablespace_name;

SELECT file_name,
       tablespace_name,
```

```

        bytes / 1024 / 1024 AS mb,
        maxbytes / 1024 / 1024 AS max_mb,
        autoextensible
FROM dba_data_files
ORDER BY tablespace_name, file_name;

SELECT df.tablespace_name,
       df.total_mb,
       df.total_mb - NVL(fs.free_mb, 0) AS used_mb,
       NVL(fs.free_mb, 0) AS free_mb,
       ROUND((df.total_mb - NVL(fs.free_mb, 0)) /
             DECODE(df.total_mb, 0, 1, df.total_mb) * 100, 2) AS pct_used
FROM (
    SELECT tablespace_name,
           SUM(bytes) / 1024 / 1024 AS total_mb
    FROM dba_data_files
    GROUP BY tablespace_name
) df,
(
    SELECT tablespace_name,
           SUM(bytes) / 1024 / 1024 AS free_mb
    FROM dba_free_space
    GROUP BY tablespace_name
) fs
WHERE df.tablespace_name = fs.tablespace_name(+)
ORDER BY pct_used DESC, df.tablespace_name;

SELECT tablespace_name,
       SUM(bytes) / 1024 / 1024 AS total_free_mb,
       MAX(bytes) / 1024 / 1024 AS largest_free_mb,
       COUNT(*) AS free_extent_count
FROM dba_free_space
GROUP BY tablespace_name;

SELECT owner,
       segment_name,
       segment_type,
       tablespace_name,
       next_extent / 1024 / 1024 AS next_extent_mb,
       extents,
       max_extents
FROM dba_segments
WHERE next_extent IS NOT NULL
ORDER BY tablespace_name, next_extent DESC, owner, segment_name;

```

Prepared fix options:

```

-- Immediate ORA-03232 fix for the checked APP_TEMP_TS case.
-- Set the extent size to 64KB because the failing allocation needed 8 blocks
-- and db_block_size was 8192 bytes.

ALTER TABLESPACE APP_TEMP_TS

```

```

DEFAULT STORAGE (
  INITIAL 64K
  NEXT 64K
  MINEXTENTS 1
  MAXEXTENTS UNLIMITED
  PCTINCREASE 0
);

ALTER TABLESPACE APP_TEMP_TS COALESCE;

-- Object-level examples only.
-- These were not executed in this incident unless a specific table/index was
-- confirmed.

-- ALTER TABLE OWNER_NAME.OBJECT_NAME
-- STORAGE (NEXT 64K PCTINCREASE 0);

-- ALTER INDEX OWNER_NAME.OBJECT_NAME
-- STORAGE (NEXT 64K PCTINCREASE 0);

```

Result:

- The immediate **ORA-03232** fix decision was based on the required allocation size:
 - existing extent/default setting: about **40KB**
 - required allocation: **8** blocks
 - block size: **8KB**
 - required extent: **64KB**
- The relevant extent setting was therefore raised to **64KB** for the incident fix.

2. Non-SYSTEM Tablespace Standardization And Capacity Additions

Prepared/checked date: **02/04/2026**

Execution date: **03/04/2026**

Reason:

- Reduce extent allocation failures and fragmented free-space risk.
- Add capacity headroom to operational tablespaces.
- Standardize future extent growth to a predictable 1MB pattern.
- This was not the immediate **ORA-03232** fix. The immediate incident fix used **64KB**; this later work standardized broader non-**SYSTEM** tablespace defaults to **1MB**.

Tablespaces handled:

- **TEMP_TS**
- **ROLLBACK_TS**
- **GENERIC_INDEX_TS**
- **TOOLS_TS**
- **SYSTEM_COMPONENT_TS**
- **GENERIC_USERS_TS**

- APP_INDEX_TS
- APP_TEMP_TS
- APP_DATA_TS

Default storage SQL:

```
ALTER TABLESPACE TEMP_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE ROLLBACK_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE GENERIC_INDEX_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE TOOLS_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE SYSTEM_COMPONENT_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE GENERIC_USERS_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE APP_INDEX_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE APP_TEMP_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);

ALTER TABLESPACE APP_DATA_TS
DEFAULT STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED PCTINCREASE
0);
```

Datafile addition SQL:

```
ALTER TABLESPACE TEMP_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\TEMP_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE ROLLBACK_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\ROLLBACK_TS_02.DBF'
```

```
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE GENERIC_INDEX_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\GENERIC_INDEX_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE TOOLS_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\TOOLS_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE SYSTEM_COMPONENT_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\SYSTEM_COMPONENT_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE GENERIC_USERS_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\GENERIC_USERS_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE APP_INDEX_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\APP_INDEX_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE APP_TEMP_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\APP_TEMP_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;

ALTER TABLESPACE APP_DATA_TS
ADD DATAFILE 'D:\ORACLE\ORADATA\LEGACYDB\APP_DATA_TS_02.DBF'
SIZE 1000M AUTOEXTEND ON NEXT 100M MAXSIZE 32767M;
```

Validation SQL:

```
SELECT tablespace_name,
       initial_extent,
       next_extent,
       pct_increase,
       status,
       contents,
       extent_management
FROM dba_tablespaces
WHERE tablespace_name IN (
    'TEMP_TS',
    'ROLLBACK_TS',
    'GENERIC_INDEX_TS',
    'TOOLS_TS',
    'SYSTEM_COMPONENT_TS',
    'GENERIC_USERS_TS',
    'APP_INDEX_TS',
    'APP_TEMP_TS',
    'APP_DATA_TS'
)
ORDER BY tablespace_name;
```

```

SELECT tablespace_name,
       SUM(bytes) / 1024 / 1024 AS total_free_mb,
       MAX(bytes) / 1024 / 1024 AS largest_free_mb,
       COUNT(*) AS free_extent_count
FROM dba_free_space
GROUP BY tablespace_name
ORDER BY tablespace_name;

```

Result:

- The check and script package were prepared on 02/04/2026.
- The non-SYSTEM tablespace default storage standardization and datafile additions were run on 03/04/2026.
- Non-SYSTEM tablespace extent defaults were standardized to the 1MB pattern.
- One additional 1000MB autoextend datafile was added per target tablespace.

3. TXN_DETAIL_A Index Creation And Statistics

Prepared/checked date: 02/04/2026

Execution date: 03/04/2026

Reason:

- Multiple high-disk_reads SQL statements were repeatedly updating APP_SCHEMA.TXN_DETAIL_A.
- The update predicates used a consistent key pattern.
- A composite non-unique index was safer than a unique index because duplicate key combinations may exist.

Top SQL check before extracting the update pattern:

```

SELECT *
FROM (
  SELECT disk_reads,
         executions,
         ROUND(disk_reads / DECODE(executions, 0, 1, executions), 2) AS
disk_per_exec,
         buffer_gets,
         rows_processed,
         address,
         hash_value,
         SUBSTR(sql_text, 1, 120) AS sql_text_sample
  FROM v$sqlarea
 WHERE
       1 = 1
 ORDER BY disk_reads DESC
)
WHERE ROWNUM <= 20;

```

High-read SQL pattern:

```
UPDATE TXN_DETAIL_A
SET QUANTITY = NVL(QUANTITY, 0) + NVL(:b1, 0)
WHERE ORG_CODE = :b2
      AND WORK_ORDER_ID = :b3
      AND LINE_ID = :b4
      AND WORK_SIZE = :b5
      AND GROUP_CODE = :b6
      AND CONTROL_STEP = :b7
      AND STEP_CODE = :b8
      AND QUALITY_CODE = :b9;
```

Index SQL:

```
CREATE INDEX APP_SCHEMA.IX_TXN_DETAIL_WORK_KEY
ON APP_SCHEMA.TXN_DETAIL_A (
    ORG_CODE,
    WORK_ORDER_ID,
    LINE_ID,
    WORK_SIZE,
    GROUP_CODE,
    CONTROL_STEP,
    STEP_CODE,
    QUALITY_CODE
)
TABLESPACE APP_INDEX_TS
STORAGE (
    INITIAL 64M
    NEXT 64M
    MAXEXTENTS UNLIMITED
    PCTINCREASE 0
);
```

Statistics SQL:

```
ANALYZE TABLE APP_SCHEMA.TXN_DETAIL_A COMPUTE STATISTICS;
ANALYZE INDEX APP_SCHEMA.IX_TXN_DETAIL_WORK_KEY COMPUTE STATISTICS;
```

Validation SQL:

```
SELECT *
FROM (
    SELECT disk_reads,
           executions,
           ROUND(disk_reads / DECODE(executions, 0, 1, executions), 2) AS
```



```

disk_per_exec,
        address,
        hash_value,
        SUBSTR(sql_text, 1, 120) AS sql_text_sample
FROM v$sqlarea
WHERE UPPER(sql_text) LIKE '%UPDATE%TXN_DETAIL_A%'
ORDER BY disk_reads DESC
)
WHERE ROWNUM <= 20;

```

Result checked on 07/04/2026:

- The index/statistics script package was prepared on 02/04/2026.
- The TXN_DETAIL_A index creation and statistics refresh were run on 03/04/2026.
- Main TXN_DETAIL_A update had about 53 disk reads / 794 executions.
- Reads per execution were about 0.07.
- The TXN_DETAIL_A tuning result was acceptable.

4. Near-2GB / 800 Processes Test Profile From test clone server

Test period: 02/04/2026

Test server: 192.0.2.51

Environment note: 192.0.2.51 is a clone of 192.0.2.50, using the same OS and the same Oracle/configuration baseline as primary production server.

File: initLEGACYDB-test-clone-tuned-near-2gb.ora

Reason:

- This profile was prepared to test a near-2GB SGA direction with a much higher process limit.
- It was tested on Oracle 8i server test clone server, which was cloned from primary production server and kept on the same OS/configuration baseline.
- The instance did not start with this profile before Windows boot mode was changed to /3GB /USERVA=2560.
- The failure reason was VM size / 32-bit user-mode virtual address pressure: the requested Oracle memory profile was too large for the normal 2GB user-mode address-space limit.

Important decision:

- Do not use this profile for normal operations.
- Do not apply this profile on production as a capacity fix.
- Keep it only as a stress/test profile to confirm whether Windows boot mode /3GB /USERVA=2560 gives enough virtual address headroom for Oracle 8i to start with a near-2GB profile.

Full near-2GB / 800 processes init file content:

```

background_dump_dest = D:\oracle\admin\LEGACYDB\bdump
compatible = 8.1.0

```

```

control_files = "D:\oracle\oradata\LEGACYDB\control03.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control01.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control02.ctl"
db_block_buffers = 220000
db_block_size = 8192
db_file_multiblock_read_count = 8
db_files = 1024
db_name = LEGACYDB
distributed_transactions = 10
global_names = TRUE
instance_name = LEGACYDB
java_pool_size = 33554432
job_queue_interval = 60
job_queue_processes = 4
large_pool_size = 16777216
log_buffer = 4194304
log_checkpoint_interval = 10000
log_checkpoint_timeout = 1800
max_dump_file_size = 10240
max_enabled_roles = 30
mts_dispatchers = "(PROTOCOL=TCP)(PRE=oracle.aurora.server.SGiopServer)"
open_cursors = 300
open_links = 4
oracle_trace_collection_name = ""
os_authent_prefix = ""
parallel_max_servers = 5
processes = 800
remote_login_passwordfile = EXCLUSIVE
service_names = LEGACYDB
shared_pool_size = 268435456
sort_area_retained_size = 524288
sort_area_size = 1048576
user_dump_dest = D:\oracle\admin\LEGACYDB\udump

```

Configured SGA estimate for this near-2GB profile:

```

db_block_buffers * db_block_size = 220000 * 8192 = 1,802,240,000 bytes = 1718.75MB
shared_pool_size = 268,435,456 bytes = 256.00MB
java_pool_size = 33,554,432 bytes = 32.00MB
large_pool_size = 16,777,216 bytes = 16.00MB
log_buffer = 4,194,304 bytes = 4.00MB

Estimated configured SGA = 1718.75 + 256.00 + 32.00 + 16.00 + 4.00
                          = about 1.98GB

```

Why it failed before /3GB /USERVA=2560:

- Normal 32-bit Windows user-mode virtual address space is about 2GB.
- This profile alone requests about **about 1.98GB** configured SGA before Oracle fixed/internal overhead.
- It also sets **processes = 800**, increasing dedicated-process memory pressure.

- Because Oracle needs additional fixed/internal memory and process address space beyond the configured component estimate, the instance could not start under the normal 2GB user-mode address-space limit.

Future use:

- Use only as a controlled boot-mode validation profile after selecting `/3GB /USERVA=2560`.
- Expected test question: "Can Oracle start with this near-2GB profile after the OS boot mode gives more user-mode VM headroom?"
- Even if it starts after `/3GB /USERVA=2560`, do not treat it as an approved production profile without separate load testing and rollback planning.

5. 1.5GB SGA / 600 Processes Test And Rollback

Execution date: `03/04/2026`

Production failure and rollback date: `07/04/2026`

Reason:

- Increase buffer cache and process capacity.
- Test whether Oracle 8i `LEGACYDB` can safely run with a larger SGA profile on Windows Server 2003.

Original stable profile: about `882MB` configured SGA / `420 processes`, init file content:

```
background_dump_dest = D:\oracle\admin\LEGACYDB\bdump
compatible = 8.1.0
control_files = "D:\oracle\oradata\LEGACYDB\control02.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control03.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control01.ctl"
db_block_buffers = 90000
db_block_size = 8192
db_file_multiblock_read_count = 8
db_files = 1024
db_name = LEGACYDB
distributed_transactions = 500
global_names = TRUE
instance_name = LEGACYDB
java_pool_size = 20971520
job_queue_interval = 10
job_queue_processes = 4
large_pool_size = 614400
log_buffer = 8388608
log_checkpoint_interval = 10000
log_checkpoint_timeout = 1800
max_dump_file_size = 10240
max_enabled_roles = 30
mts_dispatchers = "(PROTOCOL=TCP)(PRE=oracle.aurora.server.SGiopServer)"
open_cursors = 300
open_links = 4
oracle_trace_collection_name = ""
os_authent_prefix = ""
```

```
parallel_max_servers = 5
processes = 420
remote_login_passwordfile = EXCLUSIVE
service_names = LEGACYDB
shared_pool_size = 157286400
sort_area_retained_size = 524288
sort_area_size = 524288
transactions = 514
user_dump_dest = D:\oracle\admin\LEGACYDB\udump
```

Configured SGA estimate for the original stable profile:

```
db_block_buffers * db_block_size = 90000 * 8192 = 737,280,000 bytes = 703.13MB
shared_pool_size = 157,286,400 bytes = 150.00MB
java_pool_size = 20.00MB
large_pool_size = 0.59MB
log_buffer = 8.00MB
```

```
Estimated configured SGA = 703.13 + 150.00 + 20.00 + 0.59 + 8.00
                          = about 881.72MB
```

Test 1.5GB / 600 processes init file content:

```
background_dump_dest = D:\oracle\admin\LEGACYDB\bump
compatible = 8.1.0
control_files = "D:\oracle\oradata\LEGACYDB\control02.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control03.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control01.ctl"
db_block_buffers = 150000
db_block_size = 8192
db_file_multiblock_read_count = 8
db_files = 1024
db_name = LEGACYDB
distributed_transactions = 500
global_names = TRUE
instance_name = LEGACYDB
java_pool_size = 20971520
job_queue_interval = 10
job_queue_processes = 4
large_pool_size = 4194304
log_buffer = 1048576
log_checkpoint_interval = 10000
log_checkpoint_timeout = 1800
max_dump_file_size = 10240
max_enabled_roles = 30
mts_dispatchers = "(PROTOCOL=TCP)(PRE=oracle.aurora.server.SGiopServer)"
open_cursors = 300
open_links = 4
oracle_trace_collection_name = ""
```

```
os_authent_prefix = ""
parallel_max_servers = 5
processes = 600
remote_login_passwordfile = EXCLUSIVE
service_names = LEGACYDB
shared_pool_size = 268435456
sort_area_retained_size = 262144
sort_area_size = 524288
user_dump_dest = D:\oracle\admin\LEGACYDB\udump
```

Validation SQL:

```
SELECT name, value
FROM v$parameter
WHERE name IN (
    'db_block_buffers',
    'db_block_size',
    'shared_pool_size',
    'java_pool_size',
    'large_pool_size',
    'log_buffer',
    'sort_area_size',
    'sort_area_retained_size',
    'processes',
    'sessions',
    'transactions'
)
ORDER BY name;

SELECT * FROM v$sga;

SELECT resource_name,
       current_utilization,
       max_utilization,
       initial_allocation,
       limit_value
FROM v$resource_limit
WHERE resource_name IN ('processes', 'sessions', 'transactions')
ORDER BY resource_name;
```

Observed result:

- The 1.5GB / 600 processes profile was applied on 03/04/2026.
- During real production operation on 07/04/2026, the profile was not stable.
- ORA-12500: TNS:listener failed to start a dedicated server process appeared when connections/processes were above about 217.
- The profile was rolled back on 07/04/2026 to the original stable profile: about 882MB configured SGA / 420 processes.

Decision:

- Do not use 1.5GB / 600 processes on Oracle 8i primary production server.
- Continue with smaller, validated profiles because the bottleneck is likely 32-bit Windows address space and process memory, not only physical RAM.

6. Validated 981MB / 450 Processes Profile

Action date: 10/04/2026

Reason:

- The 1.5GB / 600 processes profile failed, so a smaller, safer memory/process step was needed.
- Goal: slightly improve buffer cache and process headroom while preserving 32-bit Windows memory/address-space stability.

Full 981MB / 450 processes init file content:

```
background_dump_dest = D:\oracle\admin\LEGACYDB\bdump
compatible = 8.1.0
control_files = "D:\oracle\oradata\LEGACYDB\control02.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control03.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control01.ctl"
db_block_buffers = 101520
db_block_size = 8192
db_file_multiblock_read_count = 8
db_files = 1024
db_name = LEGACYDB
distributed_transactions = 500
global_names = TRUE
instance_name = LEGACYDB
java_pool_size = 20971520
job_queue_interval = 10
job_queue_processes = 4
large_pool_size = 614400
log_buffer = 8388608
log_checkpoint_interval = 10000
log_checkpoint_timeout = 1800
max_dump_file_size = 10240
max_enabled_roles = 30
mts_dispatchers = "(PROTOCOL=TCP)(PRE=oracle.aurora.server.SGiopServer)"
open_cursors = 300
open_links = 4
oracle_trace_collection_name = ""
os_authent_prefix = ""
parallel_max_servers = 5
processes = 450
remote_login_passwordfile = EXCLUSIVE
service_names = LEGACYDB
shared_pool_size = 167772160
sort_area_retained_size = 524288
sort_area_size = 524288
```

```
transactions = 550
user_dump_dest = D:\oracle\admin\LEGACYDB\udump
```

Validation SQL:

```
SELECT * FROM v$sga;

SELECT pool, name, bytes
FROM v$sgastat
ORDER BY pool, bytes DESC;

SELECT pool, name, bytes
FROM v$sgastat
WHERE name = 'free memory'
ORDER BY pool;
```

Observed SGA result:

Fixed Size	75,804
Variable Size	212,545,536
Database Buffers	831,651,840
Redo Buffers	8,396,800

Free memory result:

shared pool free memory	about 127MB
large pool free memory	about 0.6MB
java pool free memory	about 16MB

Decision:

- Treat 981MB / 450 processes as the validated safer rollback profile.

7. 1100MB / 500 Processes Profile And TNS-12500 Evidence

Execution date: 10/04/2026

Error observed: 11/04/2026

Reason:

- Increase process capacity and buffer cache after the 981MB / 450 processes validation.
- Support higher Forms/user load, but monitor closely because this is Oracle 8i on 32-bit Windows.

Full 1100MB / 500 processes init file content:

```
background_dump_dest = D:\oracle\admin\LEGACYDB\bdump
compatible = 8.1.0
control_files = "D:\oracle\oradata\LEGACYDB\control02.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control03.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control01.ctl"
db_block_buffers = 114600
db_block_size = 8192
db_file_multiblock_read_count = 8
db_files = 1024
db_name = LEGACYDB
distributed_transactions = 500
global_names = TRUE
instance_name = LEGACYDB
java_pool_size = 20971520
job_queue_interval = 10
job_queue_processes = 4
large_pool_size = 614400
log_buffer = 8388608
log_checkpoint_interval = 10000
log_checkpoint_timeout = 1800
max_dump_file_size = 10240
max_enabled_roles = 30
mts_dispatchers = "(PROTOCOL=TCP)(PRE=oracle.aurora.server.SGiopServer)"
open_cursors = 300
open_links = 4
oracle_trace_collection_name = ""
os_authent_prefix = ""
parallel_max_servers = 5
processes = 500
remote_login_passwordfile = EXCLUSIVE
service_names = LEGACYDB
shared_pool_size = 184549376
sort_area_retained_size = 524288
sort_area_size = 524288
transactions = 610
user_dump_dest = D:\oracle\admin\LEGACYDB\udump
```

Incident evidence from listener log:

```
TNS-12500: TNS:listener failed to start a dedicated server process
TNS-12540: TNS:internal limit restriction exceeded
TNS-12560: TNS:protocol adapter error
TNS-00510: Internal limit restriction exceeded
32-bit Windows Error: 8: Exec format error
```

Observed time window:

11/04/2026 incident window

Client attempts seen:

Host: CLIENT-WORKSTATION-A
Program: D:\APP\CLIENT\Pcutbyjobwin7.exe
User: APP_USER_A
Source IP: 192.0.2.52

Host: CLIENT-WORKSTATION-B
Program: D:\APP\CLIENT\Pcutbyjob22.exe
User: APP_USER_B
Source IP: 192.0.2.53

Later healthy snapshot after the listener error had stopped:

processes	current 243	max 244	limit 500
sessions	current 240	max 242	limit 555
transactions	current 14	max 23	limit 610

Assessment:

- The 1100MB / 500 processes profile was applied on 10/04/2026; the listener error appeared during production operation in the 11/04/2026 incident window.
- The listener error was server-side: the listener received the connection request but failed to start a dedicated server process.
- The later snapshot proves the instance recovered, but it does not prove that 250 processes was the failure threshold.
- Continue monitoring the 1100MB / 500 processes profile carefully.

8. Forms Freeze Incident Capture Script

Action date: 11/04/2026

Reason:

- Forms clients became Not Responding.
- Evidence must be captured before restart or session kill, otherwise the root cause becomes unclear.

Incident capture SQL:

```
SELECT SYSDATE AS check_time FROM dual;

SELECT instance_name, host_name, version, startup_time, status
FROM v$instance;
```

```
SELECT name, open_mode, log_mode
FROM v$database;

SELECT name, value
FROM v$parameter
WHERE name IN (
    'db_block_buffers',
    'db_block_size',
    'shared_pool_size',
    'java_pool_size',
    'large_pool_size',
    'log_buffer',
    'processes',
    'transactions',
    'sort_area_size',
    'sort_area_retained_size',
    'job_queue_processes'
)
ORDER BY name;

SELECT * FROM v$sga;

SELECT pool, name, bytes
FROM v$sgastat
WHERE name = 'free memory'
ORDER BY pool;

SELECT COUNT(*) AS session_count
FROM v$session;

SELECT COUNT(*) AS process_count
FROM v$process;

SELECT status, type, COUNT(*) AS session_count
FROM v$session
GROUP BY status, type
ORDER BY status, type;

SELECT username, machine, program, status, COUNT(*) AS session_count
FROM v$session
WHERE username IS NOT NULL
GROUP BY username, machine, program, status
ORDER BY session_count DESC, username, machine;

SELECT sid, event, seconds_in_wait, state
FROM v$session_wait
WHERE event NOT LIKE 'SQL*Net message from client'
ORDER BY seconds_in_wait DESC;

SELECT l1.sid AS blocker_sid,
       l2.sid AS waiter_sid,
       l1.type,
       l1.id1,
```

```

        l1.id2,
        l1.lmode AS blocker_mode,
        l2.request AS waiter_request
FROM v$lock l1, v$lock l2
WHERE l1.id1 = l2.id1
      AND l1.id2 = l2.id2
      AND l1.block = 1
      AND l2.request > 0
ORDER BY l1.sid, l2.sid;

SELECT sid, serial#, username, status, machine, terminal, program, logon_time
FROM v$session
WHERE username IS NOT NULL
      AND status = 'ACTIVE'
ORDER BY logon_time;

SELECT s.sid,
       s.serial#,
       s.username,
       s.machine,
       s.program,
       t.start_time,
       t.used_ublk,
       t.used_urec
FROM v$transaction t, v$session s
WHERE t.ses_addr = s.saddr
ORDER BY t.used_ublk DESC;

SELECT *
FROM (
    SELECT disk_reads,
           executions,
           buffer_gets,
           rows_processed,
           SUBSTR(sql_text, 1, 120) AS sql_text_sample
    FROM v$sqlarea
    ORDER BY disk_reads DESC
)
WHERE ROWNUM <= 20;

SELECT tablespace_name,
       COUNT(*) AS free_extents,
       ROUND(SUM(bytes) / 1024 / 1024, 2) AS free_mb,
       ROUND(MAX(bytes) / 1024 / 1024, 2) AS largest_free_mb
FROM dba_free_space
GROUP BY tablespace_name
ORDER BY tablespace_name;

```

Windows checks to capture at the same time:

```

D:\oracle\admin\LEGACYDB\bdump>alert_LEGACYDB.log
Windows available memory

```

```
Windows commit/page file usage
ORACLE.EXE process/thread/handle count
TNSLSNR.EXE process/thread/handle count
OracleServiceLEGACYDB status
Listener status
Disk free space and I/O latency
```

Decision:

- Restart Oracle only after capturing Oracle, listener, alert log, and OS state if possible.

9. Windows Server 2003 /3GB /USERVA=2560 Mitigation

Plan date: 11/04/2026

Execution date: 12/04/2026

Reason:

- **ORA-12500** and **ORA-04030** together indicate process memory/address-space pressure.
- Increasing physical RAM alone does not remove the 32-bit Windows virtual address-space limit.
- Oracle's Windows database performance best-practices material recommends increasing addressable memory for 32-bit Windows Oracle processes by adding the **/3GB** switch to **boot.ini**.
- The same Oracle material also says **/USERVA=2560** helps prevent Free System Page Table Entries from falling too low, so this boot mode is used instead of a plain **/3GB** split.

Oracle best-practice source:

```
Document: Best Practices for Oracle Database Performance on Windows
Author: Christian Shay, Oracle Principal Product Manager
PDF URL:
https://www.oracle.com/technetwork/topics/dotnet/ow2011-windowsdb-bestpractices-
perf-518042.pdf?ssSourceSiteId=otnjp

Page 54 direct link:
https://www.oracle.com/technetwork/topics/dotnet/ow2011-windowsdb-bestpractices-
perf-518042.pdf?ssSourceSiteId=otnjp#page=54
Evidence: 32-bit memory best practices recommend adding /3GB to boot.ini to
increase addressable memory for the Oracle process.

Page 56 direct link:
https://www.oracle.com/technetwork/topics/dotnet/ow2011-windowsdb-bestpractices-
perf-518042.pdf?ssSourceSiteId=otnjp#page=56
Evidence: memory monitoring guidance says /USERVA=2560 in boot.ini helps prevent
Free System Page Table Entries from falling too low.
```

Boot entry added:

```
[operating systems]
multi(0)disk(0)rdisk(0)partition(1)\WINDOWS="Windows Server 2003 3GB"
/noexecute=optout /fastdetect /3GB /USERVA=2560
multi(0)disk(0)rdisk(0)partition(1)\WINDOWS="Windows Server 2003, Enterprise"
/noexecute=optout /fastdetect
```

Execution steps:

1. Back up boot.ini.
2. Add the new "Windows Server 2003 3GB" boot entry above the original boot entry.
3. Keep the original "Windows Server 2003, Enterprise" entry unchanged below it.
4. Restart the server.
5. Because this was handled remotely on a VM, do not manually select a boot entry; Windows automatically boots the first entry, so the "Windows Server 2003 3GB" entry must be the top row.
6. Verify Windows starts normally.
7. Set the Oracle init file to the near-2GB SGA test profile. This same profile had failed to start the Oracle instance before the /3GB /USERVA=2560 boot mode was applied.
8. Start Oracle and verify that the instance can start successfully after Windows is running under the /3GB /USERVA=2560 boot mode.
9. Shut down Oracle after the near-2GB start test.
10. Set the Oracle init file back to the 1100MB / 500 processes profile.
11. Start Oracle again with the 1100MB profile.
12. Verify OracleServiceLEGACYDB starts normally.
13. Verify listener status.
14. Test remote connection to 192.0.2.50:1521:LEGACYDB.
15. Monitor Forms clients under normal user load.

Post-change SQL:

```
SELECT SYSDATE AS check_time FROM dual;

SELECT instance_name, host_name, version, startup_time, status
FROM v$instance;

SELECT name, value
FROM v$parameter
WHERE name IN (
    'db_block_buffers',
    'db_block_size',
    'shared_pool_size',
    'java_pool_size',
    'large_pool_size',
    'log_buffer',
    'processes',
    'transactions'
)
```

```
ORDER BY name;

SELECT * FROM v$sga;

SELECT resource_name, current_utilization, max_utilization, limit_value
FROM v$resource_limit
WHERE resource_name IN ('processes', 'sessions', 'transactions');
```

Rollback:

Reboot into the original "Windows Server 2003, Enterprise" boot entry.

10. Post-/3GB Snapshot With 1100MB / 500 Processes

Action date: 14/04/2026

Snapshot:

Sessions from v\$session	496
Processes from v\$process	499
Configured processes limit	500
Shared pool free memory	101.09MB
Large pool free memory	0.59MB
Java pool free memory	15.36MB
Buffer cache hit ratio	37.30%
consistent gets	390,183,512
physical reads	244,772,305
physical writes	1,193,816
db file scattered read waits	37,806,438
db file sequential read waits	6,127,759
buffer busy waits	2,654

SQL used:

```
SELECT COUNT(*) AS session_count
FROM v$session;

SELECT COUNT(*) AS process_count
FROM v$process;

SELECT pool, name,
       ROUND(bytes / 1024 / 1024, 2) AS mb
FROM v$sgastat
WHERE name = 'free memory'
ORDER BY pool;

SELECT ROUND(
```

```

        (1 - (
            phy.value /
            (cur.value + con.value)
        )) * 100, 2
    ) AS buffer_cache_hit_ratio
FROM v$sysstat cur,
     v$sysstat con,
     v$sysstat phy
WHERE cur.name = 'db block gets'
     AND con.name = 'consistent gets'
     AND phy.name = 'physical reads';

SELECT name, value
FROM v$sysstat
WHERE name IN (
    'db block gets',
    'consistent gets',
    'physical reads',
    'physical writes'
)
ORDER BY name;

SELECT event,
       total_waits,
       time_waited
FROM v$system_event
WHERE event IN (
    'free buffer waits',
    'buffer busy waits',
    'db file sequential read',
    'db file scattered read'
)
ORDER BY time_waited DESC;

```

Assessment:

- /3GB /USERVA=2560 allowed the 1100MB / 500 processes profile to run under heavier load than before.
- The immediate risk became process saturation: 499 / 500.
- Shared pool free memory was acceptable in this snapshot.
- Buffer cache hit ratio was poor at 37.30%, with heavy db file scattered read, indicating broad/full-scan read pressure.

Decision:

- Do not raise processes further without OS memory and Oracle resource evidence.
- Reduce idle or leaked sessions first.
- Investigate top SQL by disk_reads.

11. High-Resource SQL Review For REPORT_DETAIL_A

Action date: 14/04/2026

Reason:

- The server showed heavy physical reads and broad scan pressure.
- A visible high-resource SQL had `hash_value = 1148157040`.

Top SQL check:

```
SELECT *
FROM (
    SELECT address,
           RAWTOHEX(address) AS address_hex,
           hash_value,
           disk_reads,
           ROUND(disk_reads / DECODE(executions, 0, 1, executions) * 8 / 1024, 2)
    AS disk_reads_per_execution_mb,
           executions,
           buffer_gets,
           rows_processed,
           SUBSTR(sql_text, 1, 120) AS sql_text_sample
    FROM v$sqlarea
    ORDER BY disk_reads DESC
)
WHERE ROWNUM <= 20
ORDER BY disk_reads DESC;

SELECT hash_value,
       sql_text
FROM v$sqltext
WHERE hash_value = 1148157040
ORDER BY piece ASC;
```

Main SQL shape reviewed:

```
SELECT
    REPORT_HEADER_A.ORG_CODE,
    AUX_HEADER_A.BATCH_TABLE,
    PROCESS_LOOKUP_A.STEP_NAME,
    AUX_DETAIL_A.PACKAGE_ID,
    REPORT_HEADER_A.DOCUMENT_ID,
    REPORT_HEADER_A.DOCUMENT_DATE,
    REPORT_HEADER_A.STEP_CODE,
    REPORT_HEADER_A.RECORD_STATUS,
    REGION_LOOKUP_A.REGION_NAME,
    REPORT_DETAIL_A.WORK_ORDER_ID,
    REPORT_DETAIL_A.LINE_ID,
    MASTER_RECORD_A.ORDER_REF,
    MASTER_RECORD_A.ITEM_STYLE,
    REPORT_DETAIL_A.ITEM_TOKEN,
    AUX_DETAIL_B.GROUP_CODE,
    AUX_DETAIL_B.ITEM_COLOR,
```



```

    AUX_DETAIL_B.ITEM_SIZE,
    REPORT_DETAIL_A.QUANTITY,
    AUX_DETAIL_B.QUANTITY,
    REPORT_DETAIL_A.EXCEPTION_QUANTITY,
    AUX_DETAIL_B.WORK_LINE_REF,
    AUX_DETAIL_B.PACKAGE_STATE,
    MASTER_RECORD_A.TARGET_DATE,
    AUX_LOOKUP_A.AUX_CODE,
    AUX_LOOKUP_A.AUX_DESC
FROM APP_SCHEMA.PROCESS_LOOKUP_A PROCESS_LOOKUP_A,
     APP_SCHEMA.REPORT_HEADER_A REPORT_HEADER_A,
     APP_SCHEMA.REPORT_DETAIL_A REPORT_DETAIL_A,
     APP_SCHEMA.AUX_DETAIL_A AUX_DETAIL_A,
     APP_SCHEMA.AUX_DETAIL_B AUX_DETAIL_B,
     APP_SCHEMA.AUX_LOOKUP_A AUX_LOOKUP_A,
     APP_SCHEMA.AUX_HEADER_A AUX_HEADER_A,
     APP_SCHEMA.MASTER_RECORD_A MASTER_RECORD_A,
     APP_SCHEMA.REGION_LOOKUP_A REGION_LOOKUP_A
WHERE REPORT_HEADER_A.ORG_CODE = 'ORG1'
AND REPORT_HEADER_A.ORG_CODE = REPORT_DETAIL_A.ORG_CODE
AND PROCESS_LOOKUP_A.ORG_CODE = REPORT_HEADER_A.ORG_CODE
AND PROCESS_LOOKUP_A.STEP_CODE = REPORT_HEADER_A.STEP_CODE
AND REPORT_HEADER_A.DOCUMENT_ID = REPORT_DETAIL_A.DOCUMENT_ID
AND REPORT_DETAIL_A.ORG_CODE = AUX_DETAIL_A.ORG_CODE
AND REPORT_DETAIL_A.ITEM_TOKEN = AUX_DETAIL_A.ITEM_TOKEN
AND REPORT_DETAIL_A.WORK_ORDER_ID = 'WORK-EXAMPLE-001'
AND REPORT_DETAIL_A.WORK_ORDER_ID = MASTER_RECORD_A.WORK_ORDER_ID
AND REPORT_DETAIL_A.LINE_ID = MASTER_RECORD_A.LINE_ID
AND MASTER_RECORD_A.REGION_CODE = REGION_LOOKUP_A.REGION_CODE
AND AUX_DETAIL_A.ORG_CODE = AUX_DETAIL_B.ORG_CODE
AND AUX_DETAIL_A.BATCH_ID = AUX_DETAIL_B.BATCH_ID
AND AUX_HEADER_A.ORG_CODE = AUX_DETAIL_B.ORG_CODE
AND AUX_HEADER_A.BATCH_ID = AUX_DETAIL_B.BATCH_ID
AND AUX_DETAIL_A.PACKAGE_ID = AUX_DETAIL_B.PACKAGE_ID
AND AUX_DETAIL_A.AUX_CODE = AUX_LOOKUP_A.AUX_CODE
AND AUX_DETAIL_B.PACKAGE_STATE <> '9'
ORDER BY AUX_DETAIL_B.GROUP_CODE,
         AUX_DETAIL_B.ITEM_SIZE,
         REPORT_HEADER_A.DOCUMENT_ID;

```

Known large table row counts:

REPORT_DETAIL_A	28,522,728
TXN_DETAIL_A	10,922,048
REPORT_HEADER_A	2,590,521
REFERENCE_TABLE_A	1,300,741
REFERENCE_TABLE_B	1,258,481
MASTER_RECORD_A	475,691

Baseline plan check:

```
EXPLAIN PLAN
SET STATEMENT_ID = 'REPORT_SQL_PUBLIC_01'
FOR
-- same report SQL shown above
SELECT ...
;

SELECT id,
       parent_id,
       operation,
       options,
       object_name,
       cost,
       cardinality,
       bytes
FROM plan_table
WHERE statement_id = 'REPORT_SQL_PUBLIC_01'
ORDER BY id;
```

Baseline plan finding:

```
REPORT_DETAIL_A full table scan cost: 37072
MASTER_RECORD_A full table scan cost: 1365
```

Existing index check:

```
SELECT i.table_name,
       i.index_name,
       i.uniqueness,
       c.column_position,
       c.column_name
FROM dba_indexes i,
     dba_ind_columns c
WHERE i.owner = 'APP_SCHEMA'
     AND i.owner = c.index_owner
     AND i.index_name = c.index_name
     AND i.table_name = c.table_name
     AND i.table_name IN (
       'REPORT_DETAIL_A',
       'MASTER_RECORD_A',
       'AUX_DETAIL_A'
     )
ORDER BY i.table_name,
         i.index_name,
         c.column_position;
```

Candidate index:

```
CREATE INDEX APP_SCHEMA.IX_REPORT_DETAIL_FILTER_KEY
ON APP_SCHEMA.REPORT_DETAIL_A
(
    WORK_ORDER_ID,
    ORG_CODE,
    DOCUMENT_ID,
    LINE_ID,
    ITEM_TOKEN
);

ANALYZE INDEX APP_SCHEMA.IX_REPORT_DETAIL_FILTER_KEY COMPUTE STATISTICS;
```

Optional follow-up indexes were not executed because the first index fixed the issue:

```
-- Not executed. Kept only as reference if the first index had not solved the
-- plan.
-- CREATE INDEX APP_SCHEMA.IX_MASTER_RECORD_WORK_KEY
-- ON APP_SCHEMA.MASTER_RECORD_A
-- (
--     WORK_ORDER_ID,
--     LINE_ID
-- );
--
-- ANALYZE INDEX APP_SCHEMA.IX_MASTER_RECORD_WORK_KEY COMPUTE STATISTICS;
--
-- CREATE INDEX APP_SCHEMA.IX_AUX_DETAIL_JOIN_KEY
-- ON APP_SCHEMA.AUX_DETAIL_A
-- (
--     ORG_CODE,
--     ITEM_TOKEN,
--     BATCH_ID,
--     PACKAGE_ID,
--     AUX_CODE
-- );
--
-- ANALYZE INDEX APP_SCHEMA.IX_AUX_DETAIL_JOIN_KEY COMPUTE STATISTICS;
```

Decision:

- `APP_SCHEMA.IX_REPORT_DETAIL_FILTER_KEY` was the index used for the fix.
- Do not create the optional follow-up indexes because the first index already fixed the issue.
- Keep the optional SQL commented for future reference only.

12. 1100MB / 550 Processes Direction

Recorded date: 17/04/2026

Reason:

- The 14/04/2026 snapshot reached 499 / 500 processes.
- Additional process headroom was needed, but it must be monitored closely because every dedicated process adds memory pressure.

Full 1100MB / 550 processes init file content:

```
background_dump_dest = D:\oracle\admin\LEGACYDB\bdump
compatible = 8.1.0
control_files = "D:\oracle\oradata\LEGACYDB\control02.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control03.ctl"
control_files = "D:\oracle\oradata\LEGACYDB\control01.ctl"
db_block_buffers = 114600
db_block_size = 8192
db_file_multiblock_read_count = 8
db_files = 1024
db_name = LEGACYDB
distributed_transactions = 500
global_names = TRUE
instance_name = LEGACYDB
java_pool_size = 20971520
job_queue_interval = 10
job_queue_processes = 4
large_pool_size = 614400
log_buffer = 8388608
log_checkpoint_interval = 10000
log_checkpoint_timeout = 1800
max_dump_file_size = 10240
max_enabled_roles = 30
mts_dispatchers = "(PROTOCOL=TCP)(PRE=oracle.aurora.server.SGiopServer)"
open_cursors = 300
open_links = 4
oracle_trace_collection_name = ""
os_authent_prefix = ""
parallel_max_servers = 5
processes = 550
remote_login_passwordfile = EXCLUSIVE
service_names = LEGACYDB
shared_pool_size = 184549376
sort_area_retained_size = 524288
sort_area_size = 524288
transactions = 671
user_dump_dest = D:\oracle\admin\LEGACYDB\udump
```

Monitoring SQL:

```
SELECT 'session_count' AS text, COUNT(*) AS cnt
FROM v$session
UNION ALL
SELECT 'process_count' AS text, COUNT(*) AS cnt
FROM v$process;
```

```
SELECT pool, name,
       ROUND(bytes / 1024 / 1024, 2) AS mb
FROM v$sgastat
WHERE name = 'free memory'
ORDER BY pool;

SELECT request_failures
FROM v$shared_pool_reserved;

SELECT namespace, gets, gethits, reloads, invalidations
FROM v$librarycache
ORDER BY reloads DESC;

SELECT name, value
FROM v$sysstat
WHERE name IN (
    'parse count (total)',
    'parse count (hard)'
);

SELECT *
FROM (
    SELECT sharable_mem / 1024 / 1024 AS mb,
           executions,
           loads,
           parse_calls,
           sql_text
    FROM v$sqlarea
    ORDER BY sharable_mem DESC
)
WHERE ROWNUM <= 20;

SELECT ROUND(
    (1 - (
        phy.value /
        (cur.value + con.value)
    )) * 100, 2
) AS buffer_cache_hit_ratio
FROM v$sysstat cur,
     v$sysstat con,
     v$sysstat phy
WHERE cur.name = 'db block gets'
AND con.name = 'consistent gets'
AND phy.name = 'physical reads';

SELECT name, value
FROM v$sysstat
WHERE name IN (
    'db block gets',
    'consistent gets',
    'physical reads',
    'physical writes'
)
```

```

ORDER BY name;

SELECT event,
       total_waits,
       time_waited
FROM v$system_event
WHERE event IN (
    'free buffer waits',
    'buffer busy waits',
    'db file sequential read',
    'db file scattered read'
)
ORDER BY time_waited DESC;

SELECT *
FROM (
    SELECT address,
           RAWTOHEX(address) AS address_hex,
           hash_value,
           disk_reads,
           ROUND(disk_reads / DECODE(executions, 0, 1, executions) * 8 / 1024, 2)
    AS disk_reads_per_execution_mb,
           executions,
           buffer_gets,
           rows_processed,
           SUBSTR(sql_text, 1, 120) AS sql_text_sample
    FROM v$sqlarea
    ORDER BY disk_reads DESC
)
WHERE ROWNUM <= 20
ORDER BY disk_reads DESC;

```

Decision:

- Keep this profile under monitoring.
- Use **primary production server** stability as the gate before applying a similar profile to another legacy server.

13. 17/04/2026 SGA / Shared Pool / Process Monitoring Result

Action date: 17/04/2026

Reason:

- Validate the Oracle 8i **primary production server** runtime state after the **1100MB** SGA / **processes = 550** direction.
- Check whether the system has process headroom, free SGA memory, shared pool pressure, and SQL area pressure.

Captured result:

```
session_count = 514
process_count = 517
```

SGA free memory:

```
java pool   free memory = 17.36MB
large pool  free memory = 0.59MB
shared pool free memory = 8.99MB
```

Shared pool component detail:

shared pool sql area	136.64MB
shared pool library cache	21.34MB
shared pool db_block_buffers	14.86MB
shared pool free memory	8.99MB
shared pool miscellaneous	4.39MB
shared pool dictionary cache	2.04MB
shared pool event statistics per sess	2.00MB
shared pool db_block_hash_buckets	1.87MB
shared pool KQLS heap	1.85MB
shared pool sessions	1.25MB

Shared pool reserved result:

```
REQUEST_FAILURES = 0
```

Library cache reload/invalidation result:

NAMESPACE	GETS	GETHITS	RELOADS	INVALIDATIONS
SQL AREA	372766	364575	6340	37
TABLE/PROCEDURE	170046	168203	205	0
TRIGGER	2379	2367	2	0
BODY	157	140	0	0
INDEX	35	5	0	0
CLUSTER	661	652	0	0

Parse counters:

```
parse count (total) = 372280
parse count (hard)  = 8991
```

Top shared-pool SQL observed:

```
SELECT REPORT_HEADER_A."ORG_CODE",
       AUX_HEADER_A."BATCH_TABLE",
       PROCESS_LOOKUP_A."STEP_NAME",
       AUX_DETAIL_A."PACKAGE_ID",
       REPORT_HEADER_A."DOCUMENT_ID",
       REPORT_HEADER_A."DOCUMENT_DATE",
       REPORT_HEADER_A."STEP_CODE",
       REPORT_HEADER_A."RECORD_STATUS",
       REGION_LOOKUP_A."REGION_NAME",
       REPORT_DETAIL_A."WORK_ORDER_ID",
       REPORT_DETAIL_A."LINE_ID",
       MASTER_RECORD_A."ORDER_REF",
       MASTER_RECORD_A."ITEM_STYLE",
       REPORT_DETAIL_A."ITEM_TOKEN",
       AUX_DETAIL_B."GROUP_CODE",
       AUX_DETAIL_B."ITEM_COLOR",
       AUX_DETAIL_B."ITEM_SIZE",
       REPORT_DETAIL_A."QUANTITY",
       AUX_DETAIL_B."QUANTITY",
       REPORT_DETAIL_A."EXCEPTION_QUANTITY",
       AUX_DETAIL_B."WORK_LINE_REF",
       AUX_DETAIL_B."PACKAGE_STATE",
       MASTER_RECORD_A."TARGET_DATE",
       AUX_LOOKUP_A."AUX_CODE",
       AUX_LOOKUP_A."AUX_DESC"
FROM   "APP_SCHEMA"."PROCESS_LOOKUP_A" PROCESS_LOOKUP_A,
       "APP_SCHEMA"."REPORT_HEADER_A" REPORT_HEADER_A,
       "APP_SCHEMA"."REPORT_DETAIL_A" REPORT_DETAIL_A,
       "APP_SCHEMA"."AUX_DETAIL_A" AUX_DETAIL_A,
       "APP_SCHEMA"."AUX_DETAIL_B" AUX_DETAIL_B,
       "APP_SCHEMA"."AUX_LOOKUP_A" AUX_LOOKUP_A,
       "APP_SCHEMA"."AUX_HEADER_A" AUX_HEADER_A,
       "APP_SCHEMA"."MASTER_RECORD_A" MASTER_RECORD_A,
       "APP_SCHEMA"."REGION_LOOKUP_A" REGION_LOOKUP_A
WHERE  REPORT_HEADER_A."ORG_CODE" = 'ORG1'
AND    REPORT_HEADER_A."ORG_CODE" = REPORT_DETAIL_A."ORG_CODE";
```

SQL-area group result:

```
WHERE sql_text LIKE 'SELECT REPORT_HEADER_A.%)'
```

```
count = 2232
```

```
shared pool memory = 123.05MB
```

Assessment:

- Process usage is high: 517 processes were already active under the processes = 550 direction.

- Shared pool free memory is low at about 8.99MB.
- There were no shared pool reserved request failures in this snapshot, so the evidence shows low free shared pool but not a reserved-pool allocation failure.
- SQL AREA usage is the main visible shared-pool pressure point.
- The `SELECT REPORT_HEADER_A...REPORT_DETAIL_A...` report query appears many times in shared pool and consumes about 123.05MB across 2232 visible SQL variants.
- This supports the earlier conclusion that SQL text churn/report SQL pressure should be reviewed together with the `REPORT_DETAIL_A` indexing direction.

Operational decision:

- Treat this as a completed 17/04/2026 monitoring snapshot.
- Continue watching process count because the instance is close to the 550 process limit.
- Do not increase `processes` again based only on process saturation.
- Next memory direction: increase `shared_pool_size` from 176MB (184549376) to 192MB (201326592) because shared pool free memory is low and SQL AREA pressure is visible.
- Investigate whether the `REPORT_HEADER_A` report SQL is generating many literal variants and whether application bind usage can be improved.
- Do not add the optional `REPORT_DETAIL_A` follow-up indexes because the first index already fixed the issue.

Next init-file change to prepare:

```
shared_pool_size = 201326592
```

Rollback And Follow-Up Rules

If `ORA-12500` or `ORA-04030` returns:

1. Capture listener log tail.
2. Capture alert log tail.
3. Capture Windows available memory, commit, and page file usage.
4. Capture ORACLE.EXE and TNSLNR.EXE process/thread/handle counts.
5. Capture `v$resource_limit`, `v$session`, `v$process`, `v$sgastat`.
6. Capture top SQL by `disk_reads`.
7. Avoid restarting before evidence capture unless production is fully blocked.

Rollback target 1:

```
db_block_buffers = 101520
shared_pool_size = 167772160
processes = 450
transactions = 550
```

Rollback target 2:

```
db_block_buffers = 90000  
shared_pool_size = 157286400  
processes = 420  
transactions = 514
```

Operational decision:

- If errors recur only near the process ceiling, reduce idle/leaked sessions first.
- If errors recur below the process ceiling, treat the cause as memory/address-space pressure and roll back to the smaller profile.
- For `REPORT_DETAIL_A`, keep the first index fix and leave the optional follow-up indexes commented unless new before/after evidence shows they are needed.